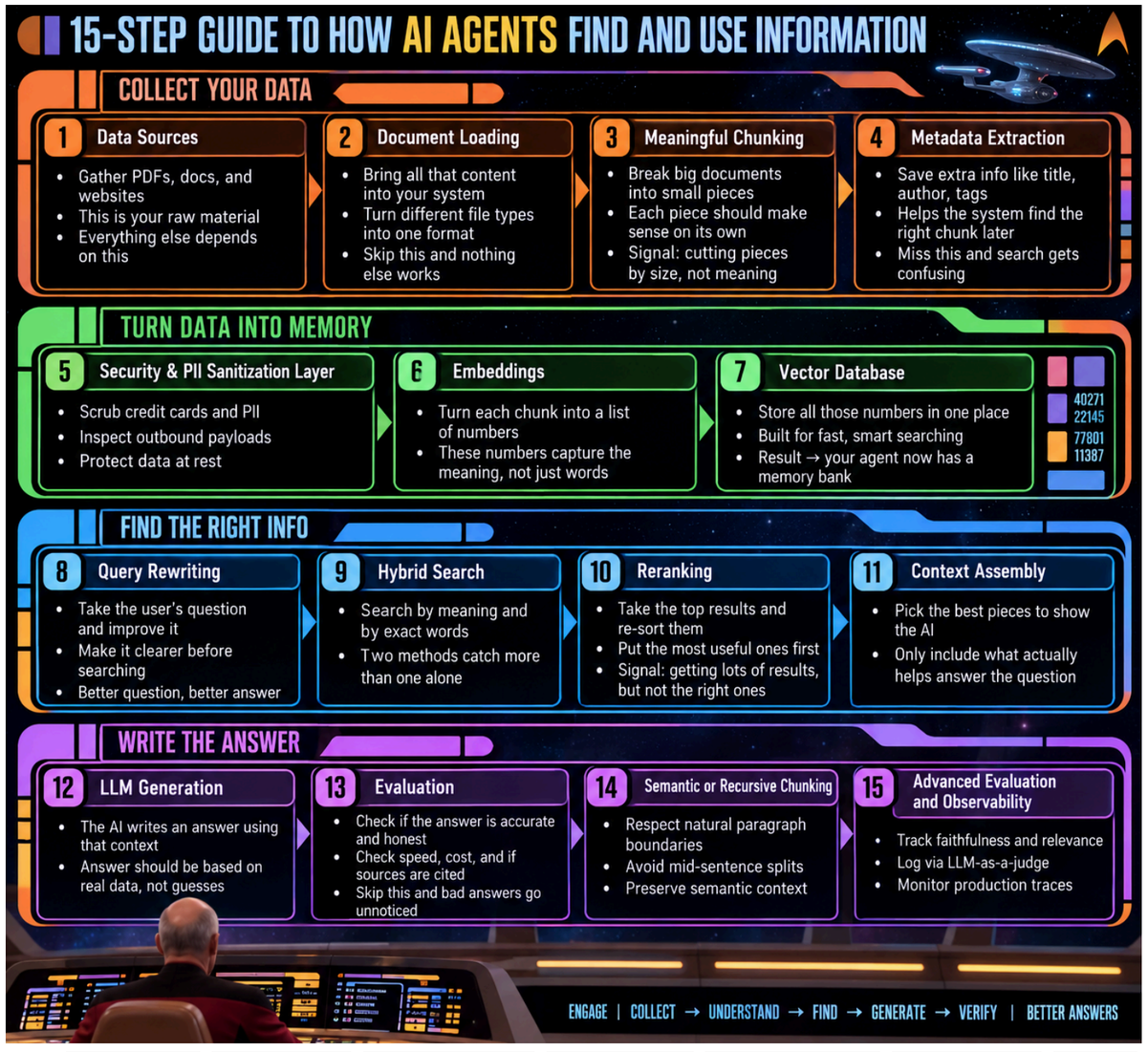
Building Enterprise-Grade AI Agents: The Complete 15-Step RAG Pipeline Architecture



Tags: Enterprise AI, RAG Pipeline, Vector Database, Hybrid Search, LLM Architecture, Data Ingestion, Security & PII, AI Agents

We have a business where we want to reference docs in RAG. For example, our business has internal policy manuals, technical specifications, and compliance PDFs stored across multiple servers, and we want to use RAG to ingest, secure, and retrieve this vital information instantly to show accurate answers to our engineering and support teams.

Here are the steps to achieve this:

Introduction

When building and scaling modern AI systems, understanding the end-to-end data lifecycle is essential for engineering reliable applications. Below is a detailed walkthrough of the complete 15-step pipeline illustrating how AI agents find, process, and utilize information based on our architectural guide.

Step 1: Data Sources

- Before any AI system can reason about internal knowledge, it requires a centralized collection of foundational files.

- **Gather PDFs, docs, and websites**: Aggregate all relevant enterprise documents, knowledge bases, and external web pages to serve as your core ingestion corpus.
- **This is your raw material**: These unstructured and semi-structured assets represent the ground truth and foundational raw materials that your system will rely on.
- **Everything else depends on this**: The downstream quality of your chunking, vector embeddings, and final LLM generation is fundamentally bounded by the completeness and accuracy of your initial data sources.

- Securing a pristine and complete set of initial source files prevents downstream processing errors and lays a solid foundation for ingestion.

Concrete Implementation Steps for Step 1:

- **Establish a Raw Data Directory**: Create a centralized storage location (such as a local `/data/raw/` directory, an Amazon S3 bucket, or a dedicated corporate document repository) to hold your source files.
- **Curate and Scope the Corpus**: Collect the specific enterprise documents required for your use case—such as PDF policy manuals, technical specifications, and internal knowledge base exports—while filtering out irrelevant or obsolete files.
- **Define Access and Permissions**: Ensure automated pipelines or ingestion scripts have secure, read-only permissions to access these source directories.

Step 2: Document Loading

- Raw binary assets and web pages must be extracted and ingested into a unified data structure.

- **Bring all that content into your system**: Ingest unstructured files from disparate storage silos and centralize them into your processing pipeline.
- **Turn different file types into one format**: Normalize diverse file extensions and formats into a single standardized text structure that downstream parsers can read.
- **Skip this and nothing else works**: Bypassing this fundamental ingestion step halts the entire pipeline, as raw binary or web assets cannot be processed directly.

- Standardizing disparate file formats into a readable text stream enables seamless passage through the rest of the pipeline.

Y.

Step 2: Document Loading Implementation Details

- **Multi-Format Loader Selection**: Choose specific loaders based on file types (e.g., `PyPDFLoader` for PDFs, `DirectoryLoader` for batch processing).
- **The Loading Execution (The Script)**: Programmatically extract raw binary or text content into memory to generate standardized document objects.
- **Basic Text Normalization & Cleaning**: Strip out corrupt characters, resolve encoding issues, and discard repetitive headers or boilerplate.
- **Initial Metadata Binding**: Associate file paths or sources to the document object to preserve lineage down the pipeline.

Code Implementation

```
from langchain_community.document_loaders import PyPDFLoader, DirectoryLoader

def load_documents(data_directory: str):
    """
    Ingests unstructured files from storage and normalizes them into a unified document structure.
    """
    # Initialize directory loader for PDF files
    loader = DirectoryLoader(
        data_directory,
        glob="./*.pdf",
        loader_cls=PyPDFLoader
    )

    # Load all documents into standard LangChain Document objects
    raw_documents = loader.load()
    print(f"Successfully loaded {len(raw_documents)} pages into standardized text format.")
    return raw_documents

# Example execution
# documents = load_documents("./data/raw/")
```

Step 3: Meaningful Chunking

- Massive documents must be broken down so they fit efficiently within processing context limits.

- **Break big documents into small pieces**: Segment massive enterprise documents into digestible, smaller segments to fit within model context windows.
- **Each piece should make sense on its own**: Ensure every generated chunk maintains local coherence and standalone context.
- **Signal: cutting pieces by size, not meaning**: Recognize that basic chunking strategies rely primarily on token size constraints rather than semantic boundaries.

- Segmenting text by size creates uniform blocks ready for subsequent metadata tagging and vectorization.

Step 3: Meaningful Chunking Implementation Details

- **Chunk Size & Overlap Definition**: Configure token or character limits (e.g., chunk size of 1000 with 200 overlap) to determine how massive enterprise documents are divided.
- **The Chunking Execution (The Script)**: Programmatically apply text splitters to segment standardized document objects into smaller, manageable blocks.
- **Token Constraint Management**: Ensure individual chunks fit comfortably within downstream embedding and LLM context window limits.
- **Standalone Coherence**: Structure splits to preserve local readability so each chunk retains clear context on its own.

Code Implementation

```
from langchain_text_splitters import RecursiveCharacterTextSplitter

def chunk_documents(raw_documents, chunk_size: int = 1000, chunk_overlap: int = 200):
    """
    Splits large standardized documents into smaller, manageable chunks for embedding.
    """
    text_splitter = RecursiveCharacterTextSplitter(
        chunk_size=chunk_size,
        chunk_overlap=chunk_overlap,
        length_function=len,
        is_separator_regex=False,
    )

    chunked_docs = text_splitter.split_documents(raw_documents)
    print(f"Successfully split {len(raw_documents)} documents into {len(chunked_docs)} chunks.")
```

```
    return chunked_docs

# Example execution
# chunks = chunk_documents(documents)
```

Step 4: Metadata Extraction

- Contextual markers must be attached to text chunks to ensure precise identification during search execution.

- **Save extra info like title, author, tags**: Attach rich contextual identifiers such as document titles, author names, creation dates, and categorical tags to every chunk.
- **Helps the system find the right chunk later**: Leverage stored metadata during filtering and hybrid queries to precisely narrow down search results.
- **Miss this and search gets confusing**: Without descriptive metadata, keyword and vector searches struggle to distinguish between similarly worded documents across different contexts.

- Embedding contextual tags directly into data chunks prevents search ambiguity and improves lookup precision.

Step 4: Metadata Extraction Implementation Details

- **Define Metadata Schema**: Determine essential key-value attributes to capture, such as document titles, authors, creation dates, and category tags.
- **The Enrichment Execution (The Script)**: Programmatically inject contextual metadata properties into each document object or chunk structure.
- **Tagging and Categorization**: Assign targeted categorical tags to group related documentation for granular database filtering.
- **Lineage Preservation**: Ensure metadata attributes remain securely attached through chunking and vector storage to enable precise retrieval queries.

Code Implementation

```
def enrich_documents_with_metadata(chunks, source_title: str, author: str, category: str):
    """
    Attaches rich contextual metadata to each text chunk for targeted retrieval filtering.
    """
    for i, chunk in enumerate(chunks):
        chunk.metadata["title"] = source_title
        chunk.metadata["author"] = author
        chunk.metadata["category"] = category
        chunk.metadata["chunk_id"] = i
```

```
    print(f"Successfully enriched {len(chunks)} chunks with custom metadata.")
    return chunks

# Example execution
# enriched_chunks = enrich_documents_with_metadata(chunks, "HR Policy Manual", "HR Department",
"Compliance")
```

Step 5: Security & PII Sanitization Layer

- Enterprise systems must enforce strict governance by filtering sensitive data before vector storage.

- **Scrub credit cards and PII**: Intercept data streams to actively redact sensitive personally identifiable information and financial records.
- **Inspect outbound payloads**: Review text segments before they reach vector storage or external model endpoints to prevent data leaks.
- **Protect data at rest**: Enforce strict governance and privacy compliance standards across all stored document segments.

- Sanitizing data safeguards privacy compliance and protects sensitive assets across internal vector repositories.

Step 5: Security & PII Sanitization Layer Implementation Details

- **PII Detection and Redaction**: Scan textual payloads for sensitive patterns such as social security numbers, credit card details, emails, and phone numbers using regex or specialized NLP models.
- **The Sanitization Execution (The Script)**: Programmatically intercept and scrub sensitive data entities from chunks before they proceed to embeddings or vector storage.
- **Outbound Payload Inspection**: Filter and review text segments to ensure no confidential corporate credentials or customer identifiers leak out.
- **Compliance Enforcement**: Maintain strict adherence to enterprise data privacy regulations (such as GDPR and HIPAA) at rest and in transit.

Code Implementation

```
import re

def sanitize_pii(chunks):
    """
    Scrubs sensitive Personally Identifiable Information (PII) like credit cards
    and social security numbers from text chunks.
    """
    # Regex patterns for common PII
    ssn_pattern = re.compile(r'\b\d{3}-\d{2}-\d{4}\b')
```

```
    credit_card_pattern = re.compile(r'\b(?:\d[ -]*?){13,16}\b')
    email_pattern = re.compile(r'\b[A-Za-z0-9._%+-]+@[A-Za-z0-9.-]+\.[A-Z|a-z]{2,}\b')

    sanitized_count = 0
    for chunk in chunks:
        original_text = chunk.page_content

        # Redact sensitive entities
        text = ssn_pattern.sub("[REDACTED_SSN]", original_text)
        text = credit_card_pattern.sub("[REDACTED_CC]", text)
        text = email_pattern.sub("[REDACTED_EMAIL]", text)

        if text != original_text:
            sanitized_count += 1

        chunk.page_content = text

    print(f"Sanitization complete. Cleaned PII across {sanitized_count} chunks.")
    return chunks

# Example execution
# safe_chunks = sanitize_pii(enriched_chunks)
```

Step 6: Embeddings

- Text blocks require numerical transformation to allow mathematical models to gauge semantic closeness.

- **Turn each chunk into a list of numbers**: Convert textual chunks into high-dimensional numerical vector arrays via embedding models.
- **These numbers capture the meaning, not just words**: Map semantic relationships and conceptual similarities into vector space rather than relying purely on exact keyword matching.

- Vector representation converts qualitative language into quantitative data capable of expressing deep semantic relationships.

Step 6: Embeddings Implementation Details

- **Model Selection**: Choose an appropriate embedding model (such as OpenAI's `text-embedding-3-small` or an open-source HuggingFace model) optimized for semantic representation.

- **The Embedding Execution (The Script)**: Programmatically convert each sanitized text chunk into high-dimensional numerical vector arrays.
- **Semantic Vector Mapping**: Capture deep contextual relationships, synonyms, and conceptual intent rather than relying solely on exact keyword matches.
- **Vector Readiness**: Structure the resulting numerical arrays for immediate persistence into a vector database.

Code Implementation

```
 from langchain_openai import OpenAIEmbeddings

def generate_embeddings(chunks):
    """
    Converts sanitized text chunks into high-dimensional numerical vectors
    that capture semantic meaning.
    """
    # Initialize the embedding model
    embeddings_model = OpenAIEmbeddings(model="text-embedding-3-small")

    # Extract text content from chunks
    texts = [chunk.page_content for chunk in chunks]

    # Generate vector embeddings for all chunks
    vectors = embeddings_model.embed_documents(texts)

    print(f"Successfully generated embeddings for {len(vectors)} chunks. Vector dimension: {len(vectors[0])}")
    return vectors

# Example execution
# vector_embeddings = generate_embeddings(safe_chunks)
```

Step 7: Vector Database

- High-dimensional numerical arrays need an optimized storage engine for fast similarity matching.

- **Store all those numbers in one place**: Persist high-dimensional embedding vectors into a specialized, highly scalable vector database.
- **Built for fast, smart searching**: Utilize specialized nearest-neighbor indexing algorithms for rapid similarity lookups.
- **Result -> your agent now has a memory bank**: Equip your AI agent with a persistent, searchable external memory repository for rapid context retrieval.

- Utilizing a dedicated vector store turns raw mathematical embeddings into a fast, queryable memory bank for agents.

Step 7: Vector Database Implementation Details

- **Vector Store Selection**: Choose a scalable vector database (such as FAISS, ChromaDB, Pinecone, or pgvector) designed for fast similarity searching.
- **The Indexing Execution (The Script)**: Programmatically ingest high-dimensional vector embeddings along with their corresponding text chunks and metadata into the database index.
- **Nearest-Neighbor Indexing**: Configure efficient indexing algorithms (such as HNSW or IVF) to enable rapid vector similarity lookups.
- **Agent Memory Repository**: Establish a persistent, queryable external memory bank that allows retrieval agents to instantly access relevant context.

Code Implementation

```
 from langchain_community.vectorstores import FAISS

def store_embeddings_in_vector_db(chunks, vectors, embeddings_model):
    """
    Persists embedding vectors and text chunks into a scalable vector database
    to create a searchable memory bank.
    """
    # Combine chunks and their vector embeddings into a FAISS vector store
    # (FAISS can also take embedding models directly to compute vectors from chunks)
    vector_store = FAISS.from_documents(
        documents=chunks,
        embedding=embeddings_model
    )

    # Save the vector store locally for future agent retrieval
    vector_store.save_local("faiss_vector_index")

    print(f"Successfully indexed and stored {len(chunks)} vectors in the FAISS database.")
    return vector_store

# Example execution
# db = store_embeddings_in_vector_db(safe_chunks, vector_embeddings, embeddings_model)
```

Step 8: Query Rewriting

- End-user prompts are often phrased colloquially and benefit from structural refinement prior to search.

- **Take the user's question and improve it**: Analyze raw user prompts and expand them to capture implied intent and context.
- **Make it clearer before searching**: Restructure ambiguous queries into precise search statements to maximize retrieval efficacy.
- **Better question, better answer**: Follow the core principle that refining the input query directly improves downstream response quality.

- Refining user prompts eliminates ambiguity and guarantees cleaner retrieval inputs.

Step 8: Query Rewriting Implementation Details

- **Analyze user intent**: Evaluate raw user queries to uncover underlying goals, implied context, and domain-specific terminology.
- **The Query Rewriting Execution (The Script)**: Programmatically pass raw prompts through a specialized prompt template and LLM to generate precise search statements.
- **Contextual Expansion**: Inject relevant synonyms and clarification markers to ensure downstream search engines capture the user's true requirement.
- **Downstream Precision**: Maximize retrieval accuracy by feeding clean, optimized queries into the hybrid search stage rather than relying on ambiguous conversational input.

Code Implementation

```
from langchain_core.prompts import ChatPromptTemplate
from langchain_openai import ChatOpenAI

def rewrite_user_query(raw_query: str) -> str:
    """
    Takes a raw, ambiguous user question and rewrites/expands it
    into a precise search statement for better retrieval.
    """
    llm = ChatOpenAI(model="gpt-4o-mini", temperature=0)

    prompt = ChatPromptTemplate.from_messages([
        ("system", "You are an AI assistant expert at query optimization for enterprise RAG systems. Rewrite the user's raw question into a clear, comprehensive, and specific search query that captures implicit intent, technical terms, and context."),
        ("human", "Raw User Query: {query}\n\nOptimized Search Query:")
    ])
```

```
    chain = prompt | llm
    response = chain.invoke({"query": raw_query})

    optimized_query = response.content.strip()
    print(f"Original Query: '{raw_query}'\nRewritten Query: '{optimized_query}'")
    return optimized_query

# Example execution
# refined_query = rewrite_user_query("how do I update my policy docs")
```

Step 9: Hybrid Search

- Combining lexical keyword matches with semantic vector lookups ensures maximum document recall.
- **Search by meaning and by exact words**: Combine dense vector similarity search with sparse keyword algorithms like BM25.
- **Two methods catch more than one alone**: Leverage the complementary strengths of semantic comprehension and exact lexical matching to maximize document recall.
- Merging dense and sparse search techniques captures both conceptual context and precise keyword references.

Step 9: Hybrid Search Implementation Details

- **Combine Dense and Sparse Retrieval**: Merge semantic vector similarity search (dense) with keyword-based algorithms like BM25 (sparse) to cover both conceptual meaning and exact term matching.
- **The Hybrid Search Execution (The Script)**: Programmatically configure an ensemble retriever to execute both search strategies simultaneously and fuse their results.
- **Maximize Recall**: Catch documents that might be missed if relying on vector distance alone (such as rare part numbers, acronyms, or specific error codes).
- **Rank Fusion**: Apply algorithms like Reciprocal Rank Fusion (RRF) to cleanly combine and score results from disparate search indexes.

Code Implementation

```
from langchain.retrievers import EnsembleRetriever
from langchain_community.retrievers import BM25Retriever
```

```
def setup_hybrid_search(chunks, vector_store):
    """
    Combines dense vector retrieval with sparse BM25 keyword search
    to maximize document recall and precision.
    """
    # 1. Initialize sparse keyword retriever (BM25) using chunk text
    bm25_retriever = BM25Retriever.from_documents(chunks)
    bm25_retriever.k = 5

    # 2. Initialize dense vector retriever from vector store
    vector_retriever = vector_store.as_retriever(search_kwargs={"k": 5})

    # 3. Combine both into an Ensemble Retriever
    ensemble_retriever = EnsembleRetriever(
        retrievers=[bm25_retriever, vector_retriever],
        weights=[0.5, 0.5]  # Equal weighting between keyword and semantic search
    )

    print("Hybrid search ensemble configured successfully.")
    return ensemble_retriever

# Example execution
# hybrid_retriever = setup_hybrid_search(safe_chunks, db)
# retrieved_docs = hybrid_retriever.invoke(refined_query)
```

Step 10: Reranking

- Initial search pools must be filtered and re-ordered using cross-encoder models to elevate top results.

- **Take the top results and re-sort them**: Pass initial retrieval candidates through advanced cross-encoder models for deep relevance scoring.
- **Put the most useful ones first**: Prioritize high-precision matches at the top of the context window to maximize signal-to-noise ratio.
- **Signal: getting lots of results, but not the right ones**: Address the common failure mode where initial retrieval yields high volume but poor precision.

- Reranking ensures that only the highest-precision segments occupy the limited prompt context window.

Step 10: Reranking Implementation Details

- **Cross-Encoder Relevance Scoring**: Pass initial retrieval candidate documents and the query together through a cross-encoder model to compute deep interaction relevance scores.

- **The Reranking Execution (The Script)**: Programmatically wrap a base retriever with a context compressor to filter, score, and re-order top search matches.
- **Signal-to-Noise Optimization**: Elevate the highest-precision matches to the absolute top of the processing window while discarding irrelevant noise.
- **Context Window Efficiency**: Prevent LLM context bloat by ensuring only the tightest, most accurate subset of documents proceeds to generation.

Code Implementation

```
from langchain.retrievers import ContextualCompressionRetriever
from langchain.retrievers.document_compressors import CrossEncoderReranker
from langchain_community.cross_encoders import HuggingFaceCrossEncoder

def setup_reranker(base_retriever, top_n: int = 5):
    """
    Wraps a base retriever with a cross-encoder reranker to deeply rescore
    and sort initial retrieval candidates by true relevance.
    """
    # Initialize a local HuggingFace cross-encoder model
    model = HuggingFaceCrossEncoder(model_name="BAAI/bge-reranker-base")

    # Configure the cross-encoder document compressor
    compressor = CrossEncoderReranker(model=model, top_n=top_n)

    # Wrap the base retriever with the compression/reranking pipeline
    compression_retriever = ContextualCompressionRetriever(
        base_compressor=compressor,
        base_retriever=base_retriever
    )

    print(f"Reranker pipeline configured to return top {top_n} precision documents.")
    return compression_retriever

# Example execution
# reranked_retriever = setup_reranker(hybrid_retriever, top_n=5)
# final_retrieved_docs = reranked_retriever.invoke(refined_query)
```

Step 11: Context Assembly

- Retrieved text segments must be cleanly formatted and packaged into a cohesive prompt context.

- **Pick the best pieces to show the AI**: Curate and package only the most relevant, reranked text segments into a structured prompt context.
- **Only include what actually helps answer the question**: Filter out redundant or noisy data to keep prompt tokens focused entirely on answering the user's prompt.

- Assembling clean context minimizes token bloat and keeps foundational model generation focused strictly on relevant facts.

Step 11: Context Assembly Implementation Details

- **Prompt Packaging and Formatting**: Curate and structure the top reranked text segments into a clean, unified context block designed specifically for prompt insertion.
- **The Context Assembly Execution (The Script)**: Programmatically join retrieved documents with clear separators and source metadata citations.
- **Token Budget Management**: Filter out redundant or noisy data to ensure prompt tokens remain focused entirely on providing actionable facts.
- **Structured Input Preparation**: Package the assembled context alongside the refined user query into the final prompt template for LLM generation.

Code Implementation

```
def assemble_context(retrieved_docs) -> str:
    """
    Formats and packages reranked document chunks into a single, cohesive
    context string with source citations for the LLM prompt.
    """
    context_blocks = []
    for i, doc in enumerate(retrieved_docs, start=1):
        source = doc.metadata.get("title", "Unknown Source")
        content = doc.page_content.strip()

        # Format each chunk with clear separation and citation metadata
        block = f"[Source {i}: {Source}]\n{content}\n"
        context_blocks.append(block)

    combined_context = "\n".join(context_blocks)
    print(f"Successfully assembled context from {len(retrieved_docs)} documents.")
    return combined_context

# Example execution
# context_string = assemble_context(final_retrieved_docs)
```

Step 12: LLM Generation

- The foundational language model synthesizes assembled context into a coherent, grounded response.

- **The AI writes an answer using that context**: Synthesize retrieved context passages through the foundational language model to generate a coherent response.
- **Answer should be based on real data, not guesses**: Ground the output strictly in retrieved enterprise documents to minimize hallucinations and speculation.

- Generating answers directly from retrieved documents guarantees factual accuracy and eliminates unverified speculation.

Step 12: LLM Generation Implementation Details

- **Response Synthesis**: Pass the assembled context and optimized user query into a powerful foundational language model to generate a precise, coherent answer.
- **The Generation Execution (The Script)**: Programmatically invoke the LLM using a strict prompt template that constrains the model to ground its response exclusively in the provided context.
- **Hallucination Prevention**: Enforce strict system instructions ensuring the AI relies only on retrieved enterprise documents rather than unverified speculation or general training memory.
- **Citation Grounding**: Configure output formatting so the model references specific source documents within its final generated response.

Code Implementation

```
from langchain_core.prompts import ChatPromptTemplate
from langchain_openai import ChatOpenAI

def generate_llm_response(query: str, context: str) -> str:
    """
    Synthesizes a grounded, accurate response using the foundational LLM
    based strictly on the assembled retrieved context.
    """
    llm = ChatOpenAI(model="gpt-4o", temperature=0)

    prompt = ChatPromptTemplate.from_messages([
        ("system", "You are an expert enterprise AI assistant. Answer the user's question accurately and professionally
using ONLY the provided context below. Include source citations (e.g., [Source 1]) when referencing facts. If the
answer cannot be found in the context, state 'I cannot find the answer in the provided documentation.'"),
        ("human", "Context:\n{context}\n\nUser Question: {query}\n\nAnswer:")
    ])
```

```
    chain = prompt | llm
    response = chain.invoke({"context": context, "query": query})

    answer = response.content.strip()
    print("LLM generation complete.")
    return answer

# Example execution
# final_answer = generate_llm_response(refined_query, context_string)
```

Step 13: Evaluation

- System output must be validated for correctness, cost, and citation traceability before delivery.

- **Check if the answer is accurate and honest**: Validate generated outputs against source citations to ensure factual correctness.
- **Check speed, cost, and if sources are cited**: Monitor operational metrics, token expenditure, and traceability to maintain system efficiency.
- **Skip this and bad answers go unnoticed**: Ensure quality gates exist so unverified or inaccurate responses do not reach end users undetected.

- Continuous evaluation safeguards system reliability by catching inaccurate or unverified responses prior to delivery.

Step 13: Evaluation Implementation Details

- **Faithfulness and Correctness Validation**: Assess generated responses against source citations to verify factual accuracy and prevent hallucinations.
- **The Evaluation Execution (The Script)**: Programmatically run evaluation metrics (such as Ragas or TruLens checks) to measure answer relevance, groundedness, and token expenditure.
- **Operational Quality Gates**: Monitor latency, cost per query, and citation traceability to maintain system efficiency in production.
- **Automated Guardrails**: Ensure unverified or inaccurate responses are caught and flagged before reaching end users.

Code Implementation

```
from langchain_openai import ChatOpenAI
from langchain_core.prompts import ChatPromptTemplate
```

```
def evaluate_rag_response(query: str, context: str, response: str) -> dict:
    """
    Evaluates the generated LLM response for factual faithfulness against
    the retrieved context and relevance to the user's query.
    """
    evaluator_llm = ChatOpenAI(model="gpt-4o-mini", temperature=0)

    prompt = ChatPromptTemplate.from_messages([
        ("system", "You are an impartial RAG evaluation judge. Rate the response on two metrics:\n1. Faithfulness
(Yes/No): Is the response derived solely from the context without unsupported claims?\n2. Answer Relevance
(Yes/No): Does the response directly address the user query?\n\nOutput in format: Faithfulness: [Yes/No] |
Relevance: [Yes/No] | Reason: [Brief explanation]"),
        ("human", "User Query: {query}\n\nContext:\n{context}\n\nGenerated Response:\n{response}\n\nEvaluation:")
    ])

    chain = prompt | evaluator_llm
    eval_result = chain.invoke({"query": query, "context": context, "response": response})

    result_text = eval_result.content.strip()
    print(f"Evaluation Results:\n{result_text}")
    return {"evaluation_report": result_text}

# Example execution
# eval_metrics = evaluate_rag_response(refined_query, context_string, final_answer)
```

Step 14: Semantic or Recursive Chunking

- Advanced document splitting preserves grammatical flow and hierarchical structure across text boundaries.

- **Respect natural paragraph boundaries**: Group text splits according to logical semantic shifts rather than arbitrary character counts.
- **Avoid mid-sentence splits**: Preserve sentence structures and grammatical flow to prevent loss of contextual meaning.
- **Preserve semantic context**: Maintain tight conceptual unity across hierarchical document structures for better downstream retrieval.

- Maintaining natural paragraph bounds prevents fragmented context and enhances semantic retrieval performance.

Step 14: Semantic or Recursive Chunking Implementation Details

- **Semantic Boundary Detection**: Group text splits according to natural paragraph breaks, section headings, and logical semantic shifts rather than arbitrary token character counts.
- **The Advanced Chunking Execution (The Script)**: Programmatically split documents using recursive or semantic chunkers to prevent mid-sentence cuts and preserve grammatical flow.
- **Context Preservation**: Maintain tight conceptual unity across hierarchical document structures so related concepts stay together in the same chunk.
- **Enhanced Retrieval Accuracy**: Improve downstream vector embedding quality by feeding structurally sound, semantically coherent text blocks into the database.

Code Implementation

```
from langchain_text_splitters import RecursiveCharacterTextSplitter

def semantic_recursive_chunking(raw_documents):
    """
    Applies advanced recursive text splitting that respects natural paragraph boundaries
    and document hierarchies to prevent fragmented context.
    """
    # Configure recursive splitter with semantic separators (paragraphs, sentences, words)
    text_splitter = RecursiveCharacterTextSplitter(
        chunk_size=800,
        chunk_overlap=150,
        separators=["\n\n", "\n", " ", ""],
        length_function=len,
    )

    semantic_chunks = text_splitter.split_documents(raw_documents)
    print(f"Successfully generated {len(semantic_chunks)} semantically structured chunks.")
    return semantic_chunks

# Example execution
# refined_chunks = semantic_recursive_chunking(documents)
```

Step 15: Advanced Evaluation and Observability

- Live production environments demand continuous trace monitoring and LLM-as-a-judge oversight.

- **Track faithfulness and relevance**: Continuously measure hallucination rates and retrieval precision using automated evaluation frameworks.

- **Log via LLM-as-a-judge**: Deploy secondary model evaluators to grade system outputs and user interactions at scale.
- **Monitor production traces**: Inspect execution steps, latency bottlenecks, and token paths across live production environments.
- Observability and automated grading tools ensure long-term stability and reliability in production deployments.

Step 15: Advanced Evaluation and Observability Implementation Details

- **Production Telemetry and Tracing**: Continuously track execution steps, latency bottlenecks, token consumption, and retrieval paths across live production environments.
- **The Observability Execution (The Script)**: Programmatically configure tracing frameworks (such as LangSmith or Arize Phoenix) to log agent runs and evaluate performance at scale.
- **Automated Judge Monitoring**: Continuously measure hallucination rates, faithfulness, and retrieval precision using automated evaluation pipelines.
- **System Stability & Governance**: Maintain long-term system reliability, audit compliance, and continuous optimization for enterprise deployments.

Code Implementation

```
import os
from langchain_core.tracers.context import tracing_v2_enabled

def configure_production_observability(project_name: str = "Enterprise-RAG-Pipeline"):
    """
    Configures environment variables and tracing wrappers for real-time
    production observability, logging, and performance monitoring.
    """
    # Set environment variables for LangSmith / LangChain tracing
    os.environ["LANGCHAIN_TRACING_V2"] = "true"
    os.environ["LANGCHAIN_ENDPOINT"] = "https://api.smith.langchain.com"
    os.environ["LANGCHAIN_PROJECT"] = project_name
    # os.environ["LANGCHAIN_API_KEY"] = "your-api-key-here"

    print(f"Production observability enabled for project: {project_name}")
    print("All agent runs, RAG retrievals, and LLM generations will now be traced.")

# Example execution
# configure_production_observability("Enterprise-Support-RAG")
```

Final Conclusion

By integrating raw data collection with rigorous multi-stage security, advanced chunking, hybrid retrieval, and continuous LLM evaluation, teams can bridge the gap between simple prototypes and enterprise-grade production architectures. Each phase builds seamlessly on the previous one, ensuring high fidelity, strict safety compliance, and maximum accuracy for modern AI agents.

Conclusion:

- **Pipeline Summary**:
- Data Sources and Document Loading gather and normalize the raw files.
- Meaningful Chunking, Metadata Extraction, Security & PII Sanitization, Embeddings, and Vector Database structure and secure the data into a memory bank.
- Query Rewriting, Hybrid Search, Reranking, and Context Assembly retrieve and refine the exact context.
- LLM Generation, Evaluation, Semantic/Recursive Chunking, and Advanced Evaluation/Observability synthesize and monitor the output.
- **Real-World Application**: In a real-world app, the user would use the AI to query company documentation, and it retrieves the precise procedural and policy details using the 15-step pipeline with RAG, and its quality is good because of rigorous PII sanitization, hybrid search, reranking, and continuous LLM-as-a-judge evaluation.

```
import os
import re
from langchain_community.document_loaders import DirectoryLoader, PyPDFLoader
from langchain_text_splitters import RecursiveCharacterTextSplitter
from langchain_openai import OpenAIEmbeddings, ChatOpenAI
from langchain_community.vectorstores import FAISS
from langchain_core.prompts import ChatPromptTemplate
from langchain.retrievers import EnsembleRetriever
from langchain_community.retrievers import BM25Retriever
from langchain.retrievers import ContextualCompressionRetriever
from langchain.retrievers.document_compressors import CrossEncoderReranker
from langchain_community.cross_encoders import HuggingFaceCrossEncoder

def run_enterprise_rag_pipeline(data_directory: str, raw_query: str):
    print("=== Step 1 & 2: Document Loading ===")
    loader = DirectoryLoader(data_directory, glob="./*.pdf", loader_cls=PyPDFLoader)
    raw_documents = loader.load()
```

```
print(f"Loaded {len(raw_documents)} pages.")

print("\n=== Step 3: Meaningful & Recursive Chunking ===")
text_splitter = RecursiveCharacterTextSplitter(
    chunk_size=800,
    chunk_overlap=150,
    separators=["\n\n", "\n", " ", ""],
    length_function=len,
)
chunks = text_splitter.split_documents(raw_documents)
print(f"Generated {len(chunks)} chunks.")

print("\n=== Step 4: Metadata Extraction ===")
for i, chunk in enumerate(chunks):
    chunk.metadata["title"] = "Internal Policy Manual"
    chunk.metadata["author"] = "Engineering & Compliance"
    chunk.metadata["category"] = "Enterprise Standards"
    chunk.metadata["chunk_id"] = i

print("\n=== Step 5: Security & PII Sanitization Layer ===")
ssn_pattern = re.compile(r'\b\d{3}-\d{2}-\d{4}\b')
credit_card_pattern = re.compile(r'\b(?:\d[ -]*?){13,16}\b')
email_pattern = re.compile(r'\b[A-Za-z0-9._%+-]+@[A-Za-z0-9.-]+\.[A-Z|a-z]{2,}\b')

sanitized_count = 0
for chunk in chunks:
    text = chunk.page_content
    text = ssn_pattern.sub("[REDACTED_SSN]", text)
    text = credit_card_pattern.sub("[REDACTED_CC]", text)
    text = email_pattern.sub("[REDACTED_EMAIL]", text)
    if text != chunk.page_content:
        sanitized_count += 1
    chunk.page_content = text
print(f"Sanitized PII across {sanitized_count} chunks.")

print("\n=== Step 6 & 7: Embeddings & Vector Database ===")
embeddings_model = OpenAIEmbeddings(model="text-embedding-3-small")
vector_store = FAISS.from_documents(documents=chunks, embedding=embeddings_model)
print("Vector database indexed and created.")

print("\n=== Step 8: Query Rewriting ===")
llm_query = ChatOpenAI(model="gpt-4o-mini", temperature=0)
query_prompt = ChatPromptTemplate.from_messages([
    ("system", "You are an AI assistant expert at query optimization for enterprise RAG systems. Rewrite the user's
raw question into a clear, comprehensive, and specific search query."),
    ("human", "Raw User Query: {query}\n\nOptimized Search Query:")
])
optimized_query = (query_prompt | llm_query).invoke({"query": raw_query}).content.strip()
print(f"Optimized Query: '{optimized_query}'")
```

```
print("\n=== Step 9: Hybrid Search ===")
bm25_retriever = BM25Retriever.from_documents(chunks)
bm25_retriever.k = 5
vector_retriever = vector_store.as_retriever(search_kwargs={"k": 5})
hybrid_retriever = EnsembleRetriever(
    retrievers=[bm25_retriever, vector_retriever],
    weights=[0.5, 0.5]
)

print("\n=== Step 10: Reranking ===")
rerank_model = HuggingFaceCrossEncoder(model_name="BAAI/bge-reranker-base")
compressor = CrossEncoderReranker(model=rerank_model, top_n=3)
reranked_retriever = ContextualCompressionRetriever(
    base_compressor=compressor,
    base_retriever=hybrid_retriever
)
final_docs = reranked_retriever.invoke(optimized_query)

print("\n=== Step 11: Context Assembly ===")
context_blocks = []
for i, doc in enumerate(final_docs, start=1):
    source = doc.metadata.get("title", "Unknown Source")
    content = doc.page_content.strip()
    context_blocks.append(f"[Source {i}: {source}]\n{content}\n")
combined_context = "\n".join(context_blocks)

print("\n=== Step 12: LLM Generation ===")
gen_llm = ChatOpenAI(model="gpt-4o", temperature=0)
gen_prompt = ChatPromptTemplate.from_messages([
    ("system", "Answer accurately using ONLY the provided context. Include source citations."),
    ("human", "Context:\n{context}\n\nUser Question: {query}\n\nAnswer:")
])
final_answer = (gen_prompt | gen_llm).invoke({"context": combined_context, "query":
optimized_query}).content.strip()

print("\n=== Step 13: Evaluation ===")
eval_llm = ChatOpenAI(model="gpt-4o-mini", temperature=0)
eval_prompt = ChatPromptTemplate.from_messages([
    ("system", "Rate Faithfulness (Yes/No) and Relevance (Yes/No). Format: Faithfulness: [Yes/No] | Relevance:
[Yes/No]"),
    ("human", "Query: {query}\nContext:\n{context}\nResponse:\n{response}")
])
eval_report = (eval_prompt | eval_llm).invoke({"query": optimized_query, "context": combined_context, "response":
final_answer}).content.strip()
print(f"Evaluation: {eval_report}")

print("\n=== Step 14 & 15: Production Observability & Pipeline Completion ===")
os.environ["LANGCHAIN_TRACING_V2"] = "true"
os.environ["LANGCHAIN_PROJECT"] = "Enterprise-RAG-Production"
print("Pipeline executed successfully and traced.")
```

```
    return final_answer

# Execute Pipeline
# final_result = run_enterprise_rag_pipeline("./data/raw/", "How do I update my policy docs?")
```